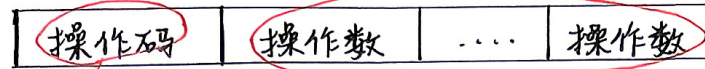


计算机的指令系统就是指该计算机能够执行的全部指令的集合

16位8086指令系统是整个Intel 80x86系列微处理器指令系统的基础

指令格式

指令由操作码和操作数2部分组成



说明计算机执行何操作
(传送/运算/移位/跳转...)

指令执行的参与者,即各种操作的对象
(有些指令没有操作数,大多有1~2个,有的甚至有3~4个)
- / = /
三 / ... 地址指令

操作码 操作数1, 操作数2 ; 注释

目的操作数(dest), 源操作数(src), 表示参与指令操作的一个对象

不仅可作指令操作的一个对象,还可存放指令操作的结果

寻址方式 { 与数据有关的寻址方式: 立即数寻址方式、寄存器寻址方式、存储器寻址方式(5)
与转移地址有关的寻址方式: 段内转移(仅变偏移地址)、段间转移(变段地址、偏移地址)
寻址: 寻找操作数的地址

寻址方式: 寻找操作数的方法

由指令指定操作数的位置, 给出地址信息, 在执行时需根据地址信息找到需要的操作数

立即数: 跟在指令操作码之后的操作数

寄存器操作数: 存放在CPU内部寄存器中的操作数

存储器操作数: 存放在内存存储器中的操作数(绝大多数情况)

与数据有关的寻址方式

以 mov 指令为例：

立即寻址	MOV AX, 3069H
寄存器寻址	MOV AL, BH
直接寻址	MOV AX, [2000H]
寄存器间接寻址	MOV AX, [BX]
寄存器相对寻址	MOV AX, COUNT[SI] 或 MOV AX, [SI + COUNT]
基址变址寻址	MOV AX, [BP][DI]
相对基址变址寻址	MOV AX, MASK[BX][SI] 或 MOV AX, [SI + BX + MASK]

① 立即寻址方式

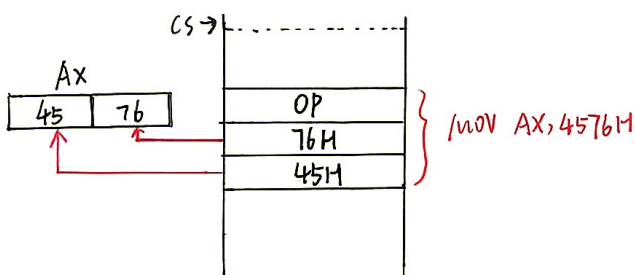
用8116位二进制补码表示的有符号数

操作数作为指令的一部分直接写在指令中 (这种操作数称为立即数)

指令
操作数 n

指令下一单元的内容就是操作数, 操作数存放在存储器

★ 常用于给寄存器和存储单元赋值, 多以常量形式出现



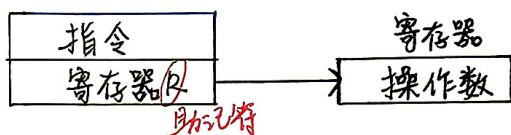
例. MOV AX, 4576H 执行后 (AX) = ?

解: 4576H → AX

源操作数为立即寻址方式, 立即数是 4576H, 存放在指令的下一单元

② 寄存器寻址方式

操作数存放在寄存器或把目标操作数存入寄存器, 在指令中指出寄存器



8位: AH, AL, BH, BL, CH, CL, DH, DL ...

16位: AX, BX, CX, DX, SI, DI, SP, BP, CS, DS, ES, SS ...

★ 减少读/写存储器单元的次數, 具有較快的執行速度

例. MOV AX, 1234H 执行后 (AX) = ?
 MOV BX, 5678H (BX) = ?
 ADD AX, BX

解: 1234H → AX
 5678H → BX
 (BX) + (AX) → (AX)

(AX) = 68ACH (BX) = 5678H

③ 存储器寻址

以下各寻址方式的操作数都在除代码段外的存储区中，它们体现了求有效地址的不同途径

(操作数的物理地址 = 段基址左移4位 + 偏移地址)

在默认或用段超越前缀指定的段寄存器中 即有效地址

8086/8088指令中有效地址(EA)由以下3部分组成:

有效地址 = 基址 + 变址 + 位移量
EA = BX/BP + SI/DI + 8/16位位移量

基址: 存放在基址寄存器(BX、BP)中的内容, 常用于指向数据段中数组/字符串首地址
变址: 存放在变址寄存器(SI、DI)中的内容, 常用于指向数组中某元素/字符串某字符
位移量: 存放在指令中的1个8位/16位的数, 是1个地址而非立即数

当基址用BP时, 默认使用SS提供段基地址; 其它情况默认使用DS提供段基地址
可用段超越前缀改变:

8086/8088允许使用跨越段前缀来改变系统指定的默认段:

表3.2 段寄存器使用的基本约定

访问存储器类型	默认段寄存器	可指定段寄存器	段内偏移地址来源
取指令码	CS	无	IP
堆栈操作	SS	无	SP
串操作源地址	DS	CS、ES、SS	SI
串操作目的地址	ES	无	DI
BP 用作基址寄存器	SS	CS、DS、ES	根据寻址方式求得有效地址
一般数据存取	DS	CS、ES、SS	根据寻址方式求得有效地址

有3种情况不允许使用跨越段前缀:

- ① 指令必须在代码段CS中
- ② PUSH指令的源操作数和POP指令的目的操作数必须使用SS段
- ③ 串处理指令的目的串必须使用ES段

依据有效地址中含有的成分不同, 分别构成不同的寻址方式:

直接寻址方式

寄存器直接寻址方式

寄存器间接寻址方式

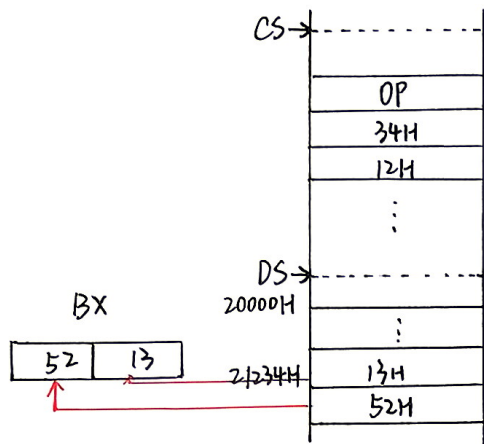
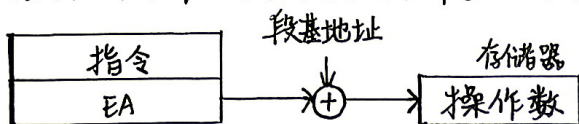
基址变址寻址方式

相对基址变址寻址方式

1) 直接寻址

操作数存放在内存中，在指令中直接给出该操作数的有效地址，

指令下一个字单元的内容是操作数的偏移地址EA



例. 执行指令 `MOV BX, [1234H]`，设 $(DS) = 2000H$

执行后 $(BX) = ?$

解:

$$\begin{array}{r} DS: \quad 2000H \\ + \quad 1234H \\ \hline 21234H \end{array}$$

一般操作数存放在数据段中，物理地址由DS和指令中的EA直接形成
但也可能使用段超越前缀: `MOV ES: [1234H], AX`

注意: 在程序中，直接地址通常用内存变量名表示。例如: `VARW` (内存字变量)
`MOV AX, VARW` $\Leftrightarrow$ `MOV AX, [VARW]` 都是直接寻址

变量指示内存中的数据，变量名具有地址的属性

变量定义:

`WVAR DW 1234H` ; 16位变量, 初值1234H

假设它被分配到内存中某位置，偏移地址是10H

单独引用变量名 - 直接寻址，代表地址里存放的数据:

`MOV AX, WVAR` ; 等价于 `MOV AX, [0010H]` 功能: $1234H \rightarrow AX$

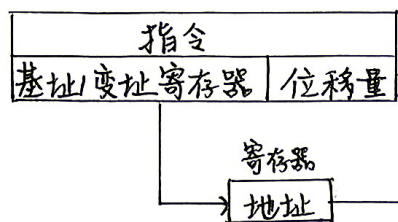
相对寻址，代表它的偏移地址值:

`MOV AX, [DI+WVAR]` ; 等价于 `MOV AX, [DI+0010H]` 功能: $WVAR[DI] \rightarrow AX$

($[DI+WVAR] \rightarrow AX$)

2) 寄存器相对寻址方式

操作数在存储器中，有效地址是一个基址寄存器 (BX/ BP) 或变址寄存器 (SI/ DI) 中的内容与指令中的 8/16 位位移量之和



MOV AX, [SI+06H]

MOV AX, 06H[SI]

$\text{DS:[SI+06H]} \rightarrow \text{AX}$

[地址] 取这个地址中的值 (寄存器名/地址) 当前存的值

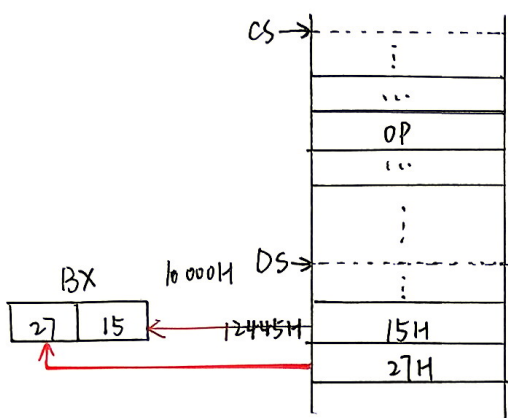
例. MOV BX, [SI+100H] , $(\text{DS})=1000\text{H}$, $(\text{SI})=2345\text{H}$,
内存单元 12445H 的内容是 2715H, 执行后 BX 的值?

解:

$$\text{EA} = (\text{SI}) + 100\text{H} = 2345\text{H} + 100\text{H} = 2445\text{H}$$

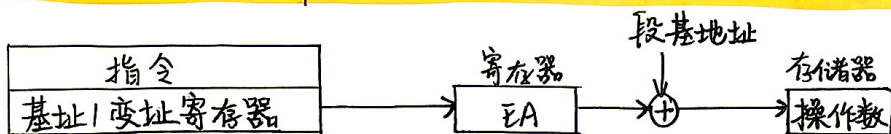
$$\text{PA} = (\text{DS}) * 16 + \text{EA} = 1000\text{H} * 16 + 2445\text{H} = 12445\text{H}$$

把从物理地址为 12445H 开始的一个字的值传送给 BX



3) 寄存器间接寻址方式

操作数在存储器中，有效地址由 BX/ BP/ SI/ DI 之一指定



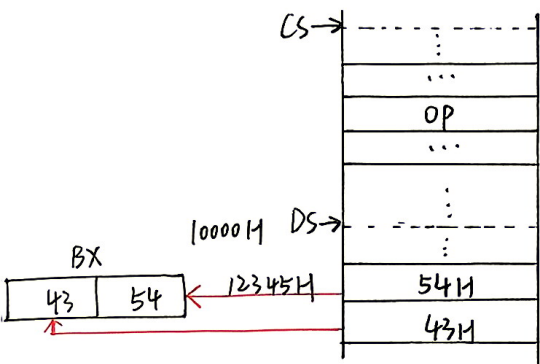
例. MOV BX, [DI] , $(\text{DS})=1000\text{H}$, $(\text{DI})=2345\text{H}$,

内存单元 12345H 的内容是 4354H, 执行后 BX 的值?

解:

$$\text{PA} = (\text{DS}) * 16 + (\text{DI}) = 1000\text{H} * 16 + 2345\text{H} = 12345\text{H}$$

把从物理地址为 12345H 开始的一个字的值传送给 BX



在不使用段超越前缀的情况下:

若有效地址用 BP 指定, 则默认段寄存器为 SS, $\text{PA} = (\text{SS}) * 16 + \text{BP}$

若有效地址用 BX/ SI/ DI 指定, 则默认段寄存器为 DS, $\text{PA} = (\text{DS}) * 16 + \text{BX/ SI/ DI}$

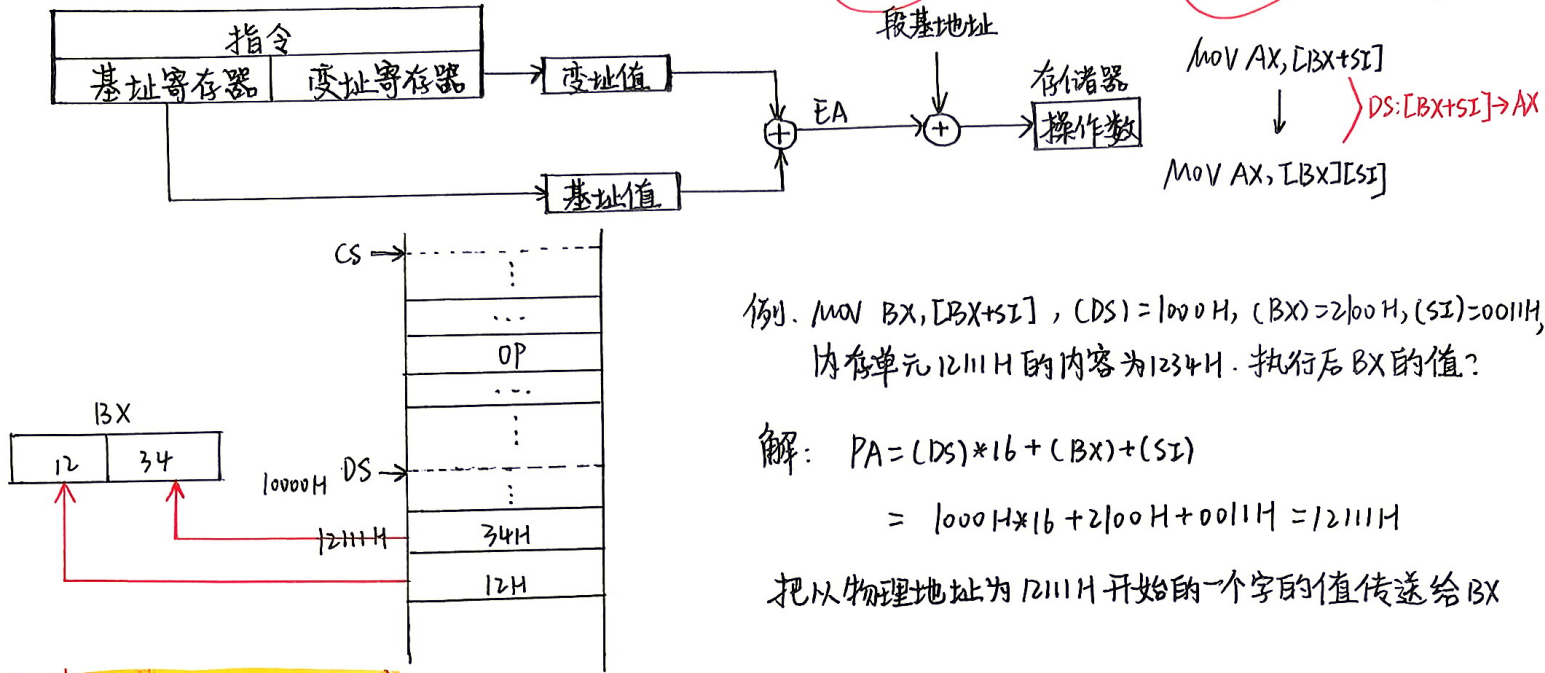
使用案例:

MOV AX, DS:[BP]

MOV CX, SS:[SI]

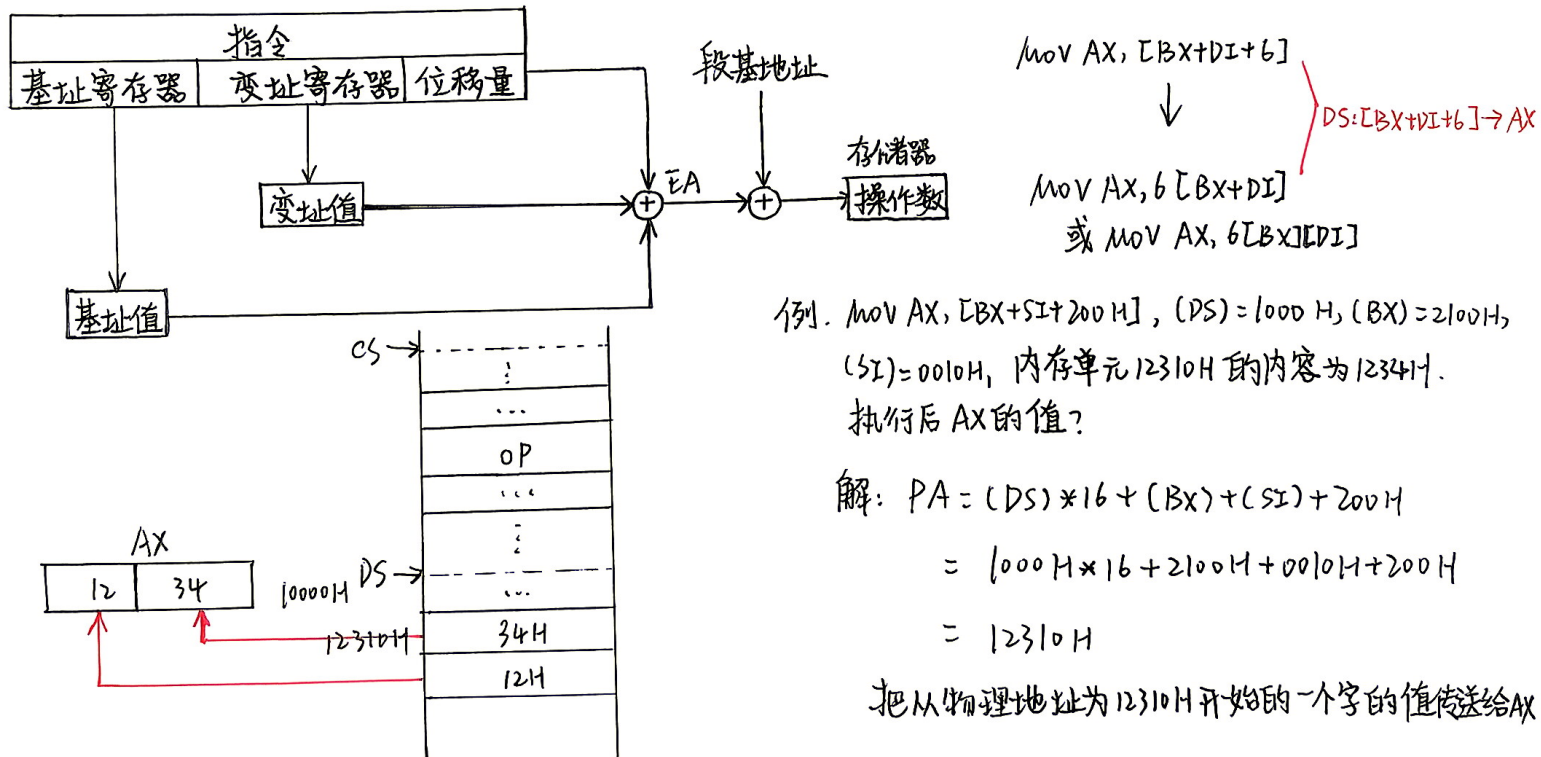
4) 基址变址寻址方式

操作数在存储器中，有效地址是一个基址寄存器 (BX/BP) 与一个变址寄存器 (SI/DI) 的内容之和



5) 相对基址变址寻址方式

操作数在存储器中，有效地址是一个基址寄存器 (BX/BP)、一个变址寄存器 (SI/DI) 的内容与指令中 8 位/16 位偏移量之和



与转移地址有关的寻址方式

用以确定转移指令及转子 (call) 指令的转向地址

转移地址是由各种寻址方式得到的有效地址和段地址相加而成

有效地址存入 IP 寄存器中, 段地址指定为 CS 段寄存器内容

① 段内寻址

段内直接寻址 `JMP NEAR PTR NEXT`

段内间接寻址 `JMP TABLE[BX]`

② 段间寻址

段间直接寻址 `JMP FAR PTR NEXT`

段间间接寻址 `JMP DWORD PTR [BX]`

段内转移:

⇒ **近转移 (near)** ^{→ ±32KB} 在当前代码段 64KB 范围内转移
无需更改 CS 段地址, 只改变 IP 偏移地址即可

⇒ **短转移 (short)** 在段内 -128 ~ 127 范围内转移 (可用 1 字节表达)

段间转移:

⇒ **远转移 (far)** 从当前代码段跳到另一代码段, 范围 1MB
需更改 CS 段地址和 IP 偏移地址 例如 CS:IP
目标地址必须用一个 32 位数表达 (32 位远指针), 它是 逻辑地址

- 实际编程时, 汇编程序会根据目标地址的属性而自动处理成短/近/远转移
- 我们可用操作符将 short, near ptr 或 far ptr 强制成为需要的转移类型

`JMP label` ; 段内转移, 直接寻址 → label 标记与当前指令的地址差
 $IP + \text{位移量} \rightarrow IP$

`JMP r16/m16` ; 段内转移、间接寻址 $r16/m16 \rightarrow IP$ 从寄存器或内存单元取址

`JMP far ptr label` ; 段间转移、直接寻址 偏移地址 → IP, 段地址 → CS 跳至其它代码段的 label 标记的位置

`JMP far ptr mem` ; 段间转移、间接寻址 $[mem] \rightarrow IP, [mem+2] \rightarrow CS$ 从内存单元获取目标
的段地址、偏移地址

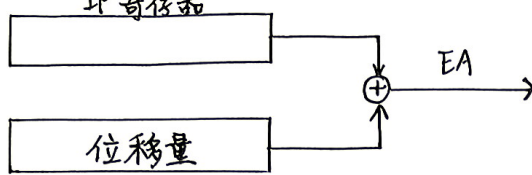
无条件转移指令 `JMP` 使程序转向指定的 目标地址 处执行
↓
label

① 段内转移

程序在同一代码段内, 仅改变IP寄存器的值而不改变CS寄存器的值所发生的转移

1) 段内直接寻址

转向的有效地址 = 当前(IP) + 位移量 (8 bit/16 bit)



• JMP SHORT QUEST 短跳转

QUEST表示转移的符号地址, 操作符SHORT表示8位带符号数(80H~7FH 即-128~+127)
只能相对于当前IP(转移指令的下一条指令的首地址)做-128~+127范围内的跳转

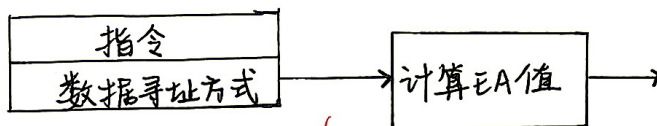
• JMP NEAR PTR PROGA 近跳转

PROGA表示转移的符号地址, 操作符NEAR PTR表示16位带符号数(8000H~7FFFH 即-32768~+32767)
只能相对于当前IP所指位置做-32768~+32767范围内的跳转

2) 段内间接寻址

转向的有效地址是一个寄存器或存储单元的内容,

可用除立即数以外任一寻址方式得到, 用所得有效地址取代IP寄存器的内容



C) 里放偏移地址

例. 已知 $TABLE = 20A2H$, $(BX) = 1256H$, $(SI) = 528EH$, $(DS) = 2000H$, $(232F8H) = 3280H$,
 $(264E4H) = 2450H$ (1) 里放物理地址

JMP BX

; $(IP) = 1256H$

JMP WORD PTR TABLE[BX]

; $(DS) \times 16 + TABLE + (BX) = 232F8H$ $(IP) = 3280H$

JMP WORD PTR [BX][SI]

; $(DS) \times 16 + (BX) + (SI) = 264E4H$ $(IP) = 2450H$

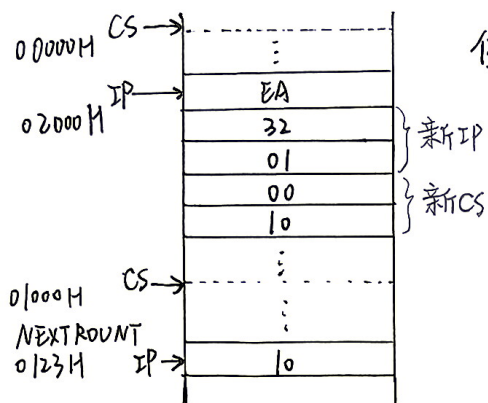
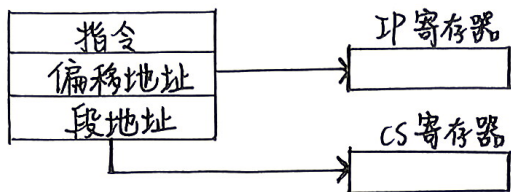
△★

② 段间转移

当程序从一个代码段转移到另一个代码段时，同时改变IP寄存器和CS寄存器的值

1) 段间直接寻址

用指令中提供的转向段地址和偏移地址取代IP和CS

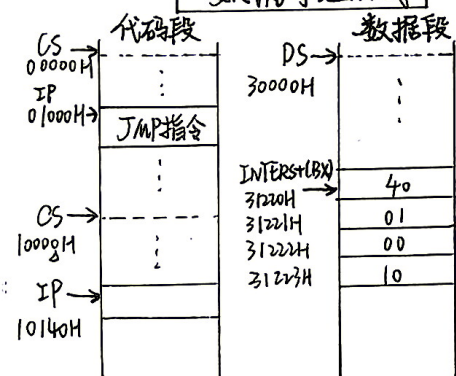


例. 已知 $(CS) = 0000H$, $(IP) = 2000H$

JMP FAR PTR NEXTROUT

2) 段间间接寻址

用存储器中的2个相继的字的内容取代CS和IP，存储单元的地址可用存储器寻址的方式得到



例. JMP DWORD PTR [INTER5 + BX]

已知 $(DS) = 3000H$, $(BX) = 1200H$, $INTER5 = 0020H$,

解: $PA = (DS) * 16 + INTER5 + (BX)$
 $= 30000H + 0020H + 1200H = 31220H$

执行后: $(CS) = 1000H$, $(IP) = 10140H$,

$(31220H) = 40H$, $(31221H) = 01H$, $(31222H) = 00H$, $(31223H) = 10H$

$(\text{新CS}) * 16 + (\text{新IP})$
 $= 10008H + 0140H = 10140H$

$$\begin{cases} 31220H = 40H \\ 31221H = 01H \\ 31222H = 00H \\ 31223H = 10H \end{cases}$$

80x86指令系统分为6组:

- ① 数据传送类指令 ② 算术运算类指令 ③ 逻辑运算类指令
④ 字符串处理指令 ⑤ 控制转移指令 ⑥ 处理机控制及杂项操作指令

注意:

- 指令操作数的寻址方式
- 指令对标志位的影响、标志位对指令的影响
- 指令的基本功能(对可完成相同功能的指令,选取执行时间短的)

① 数据传送类指令

把数据、地址或立即数传送到寄存器、存储器或端口号寄存器

{ 通用数据传送: MOV、XCHG、PUSH/POP
累加器专用传送: IN/OUT、XLAT
地址传送: LEA、LDS、LES
标志寄存器传送: LAHF、SAHF、PUSHF/POPF
类型转换指令: CBW、CWD

除标志寄存器传送指令外,均不影响标志位

(1) MOV

MOV 目的操作数,源操作数

把源操作数的值传给目的操作数

目的操作数可以是(寄存器)或(存储器),源操作数可以是(寄存器)或(存储器)或立即数 ↗ 8/16位

在存储单元是该指令的一个操作数时,该操作数的寻址方式可以是任一种存储单元寻址方式

例如

MOV CL, 4 ↗ 字母开头的常数 4 → CL 字节传送

MOV DX, 0FFH 加 前导0 ; 00FFH → DX 字传送

MOV [BX], AL ; AL → [BX] 字节传送

MOV DS, AX ; AX → DS 字传送

MOV 寄存器 AL, 寄存器指向的内存单元中的数据 [BX] ; DS:[BX] → AL

MOV DX, [BP+4] ; SS:[BP+4] → DX

MOV AX, DS ; DS → AX

把8位寄存器AL中的数据传送到BX指向的内存单元中

把DS段中BX寄存器指向的内存单元中的数据传送到AL中

[]: 把其中内容当作偏移地址,访问该地址对应的内存单元数据

注意:

- 2个操作数应类型一致, 同位8位/16位
 - 不应同为段寄存器
 - 不应同为存储单元 (MOV VARA, VARB $\times$ VARA和VARB是同数据类型的内存变量)
- CS不能作目的操作数, 立即数不能作目的操作数
- 立即数不能直接传给段寄存器
- 指令指针IP不可作MOV指令的操作数

非法指令

⇒ 两操作数类型不同 (应同为字量或字节量)

~~MOV AX, 050AH~~ $\rightarrow$ MOV AX, 050AH

~~MOV SI, DL~~ $\rightarrow$ MOV DH, 0
MOV SI, DX

⇒ 无法确定是字节量还是字量操作

~~MOV [BX+SI], 255~~ $\rightarrow$ MOV BYTE PTR [BX+SI], 255 ; 把255作8位数据传到 [BX+SI] 对应的1个内存字节单元
 $\rightarrow$ MOV WORD PTR [BX+SI], 255 ; 把255作16位数据传到 [BX+SI] 对应的1个内存字单元

⇒ 两操作数都是存储器操作数 (除串操作指令外)

~~MOV BUF2, BUF1~~ $\rightarrow$ MOV AX, BUF1 ; BUF1, BUF2是2个字变量
MOV BUF2, AX
 $\rightarrow$ MOV AL, BUF1 ; BUF1, BUF2是2个字节变量
MOV BUF2, AL

⇒ 段寄存器操作的限制 (能直接对段寄存器操作的指令只有MOV等极个别的)

~~MOV DS, ES~~ $\rightarrow$ MOV AX, ES
MOV DS, AX

~~MOV DS, 100H~~ $\rightarrow$ ~~MOV AX, 100H~~
~~MOV DS, AX~~ $\rightarrow$ ~~指令存在但不可执行~~

~~MOV DS, [SI]~~ $\rightarrow$ ~~指令存在但不可执行~~

(2) XCHG 交换指令

XCHG 目的操作数, 源操作数

可在寄存器之间或通用寄存器与存储器之间进行一字节或1个字的数据交换

段寄存器和立即数不能做其操作数

例. 假设 (AL) = 2AH, (204DH) = 5BH, 则 XCHG AL, [204DH] 执行后

先进后出

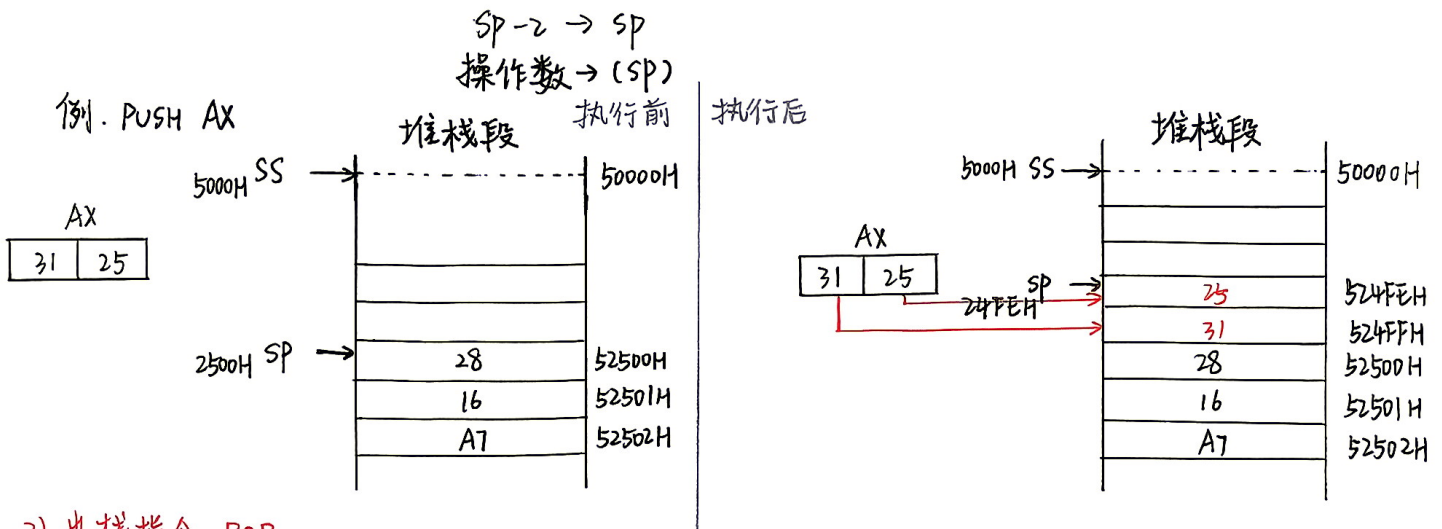
(AL) = 5BH, (204DH) = 2AH

(3) 堆栈指令

1) 进栈指令 PUSH

PUSH 源操作数

堆栈指针 - 2H, 将寄存器/段寄存器/存储器中的一个字数据压入堆栈



2) 出栈指令 POP

POP 目的操作数

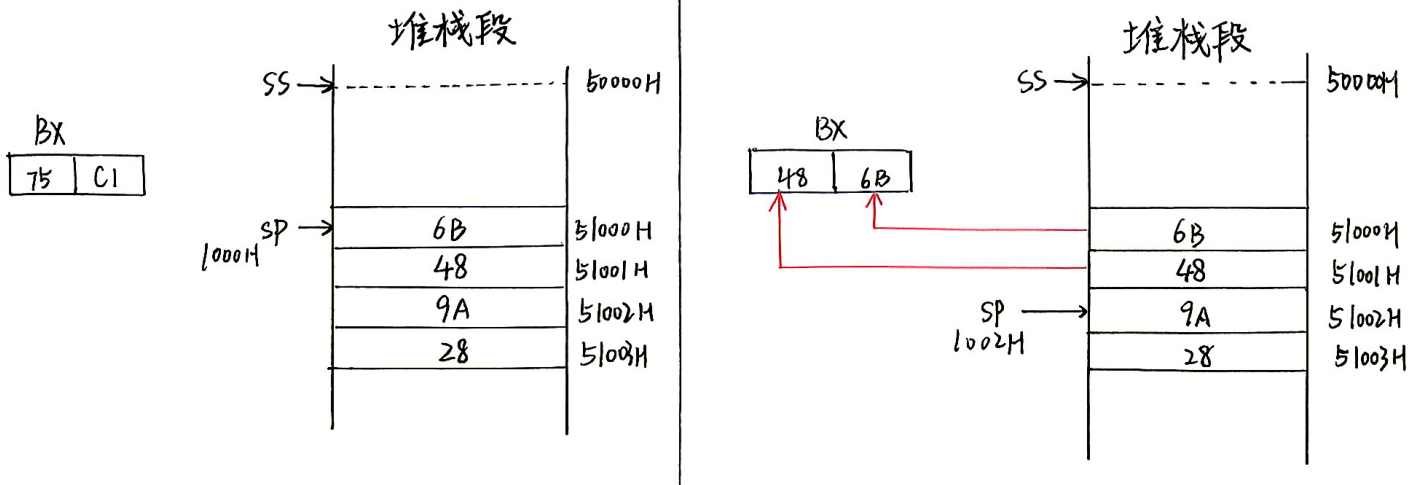
将栈顶元素弹出送至某-寄存器/段寄存器(除CS)/存储器, 堆栈指针 + 2H

(SP) → 操作数

SP + 2 → SP

例 POP BX

执行前 执行后



堆栈操作的特点:

- 堆栈操作的单位是字, 进栈、出栈只对字量
- 字量数据从栈顶压入和弹出时, 都是“低对低, 高对高”
- 堆栈操作遵循先进后出的原则,
但是可以用存储器寻址方式随机存取堆栈中的数据
- 堆栈段是程序中不可或缺的一个内存区, 常用来临时存放数据、传递参数、保存与恢复寄存器
- 不影响标志位
- 不能用立即寻址方式 ~~PUSH 1234H~~
- 目的操作数不能是CS ~~POP CS~~
- 所有寄存器进栈指令 PUSH A

次序: AX, CX, DX, BX, SP, BP, SI, DI
(SP) - 16 → (SP)

所有寄存器出栈指令 POP A

次序: DI, SI, BP, SP, BX, DX, CX, AX

△
只修改了指针但 (SP) + 16 → (SP)

不会把值送到SP寄存器, 否则就出岔子了!

(4) 累加器专用传送指令

仅限于使用累加器 AX 或 AL 传送信息

1) 输入指令 IN

从指定的外设寄存器取信息送入累加器

长格式:

IN AL, 端口号 ; 字节
或 IN AX, 端口号 ; 字 (IN AX, 201H

AX	
21H 中数据	201H 中数据

)

短格式:

MOV DX, 端口号
IN AL, DX ; 字节
或 IN AX, DX ; 字 (IN AX, 2F8H

AX	
2F9H 中数据	2F8H 中数据

)

若某输入设备的端口地址在 0~255 (00H~0FFH) 之间, 则可在 IN 指令中直接给出

2) 输出指令 OUT

把累加器内容送往指定的外设寄存器

长格式:

OUT 端口号, AL ; 字节
或 OUT 端口号, AX ; 字

短格式:

MOV DX, 端口号
OUT DX, AL ; 字节
OUT DX, AX ; 字

- 8086/8088 中, 所有 I/O 端口与 CPU 间的通信均由 IN、OUT 指令完成

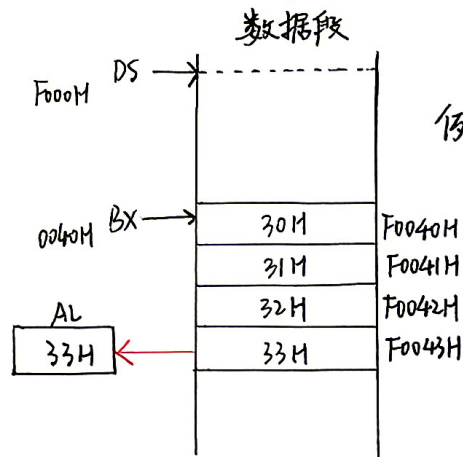
{ IN 指令完成从 I/O 端口到 CPU 的数据传送 (输入)
OUT 指令完成从 CPU 到 I/O 端口的数据传送 (输出)

- 外部设备最多有 2^{16} 个 I/O 端口 (0000H~0FFFFH), 前 2^8 个 I/O 端口 (00H~0FFH) 可在指令中指定, 可以使用“长格式”, 从 256 位后的端口只能使用“短格式”, 先将端口号放入 DX 寄存器再用 IN/OUT 指令
- 端口号或 DX 的内容都是地址, 但传送的是端口的信息.
- 使用短格式时, DX 中内容就是端口号本身

3) XLAT 换码指令

转换指令有两个隐含操作数 BX 和 AL

把内存字节数组首地址为 BX、下标为 AL 的元素的值传递给 AL



$$((BX) + (AL)) \rightarrow AL$$

例. `MOV BX, offset table` ; (BX) = 0040H

`MOV AL, 3`

`XLAT`

执行后 (AL) = ?

解: $!! (BX) + (AL) = 0040H + 3H = 0043H$

$$PA = (DS) * 16 + 0043H = F000H * 16 + 0043H = F0043H$$

$$(F0043H) = 33H$$

$$\therefore (AL) = 33H$$

(5) 地址传送指令

将地址送到指定寄存器

1) 有效地址送寄存器指令 LEA

LEA 寄存器, 源操作数

将源操作数的有效地址送到指定寄存器中

常用来建立串操作指令所需寄存器指针

寄存器须是16位通用寄存器, 源操作数须是内存操作数(非立即数, 非寄存器中数)

2) 指针送寄存器及相应段寄存器指令 LDS 和 LES

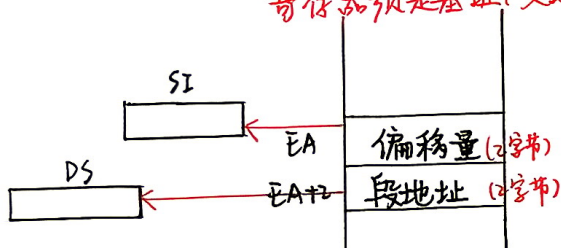
⇒ LDS 寄存器, 源操作数

完成一个地址指针的传送

地址指针包括段地址部分和偏移量部分

指令将段地址(EA+2)送入DS, 偏移量部分(EA)送入16位基址/变址寄存器

寄存器须是基址/变址寄存器, 源操作数须是内存操作数



例. LDS SI, [BX]

把BX所指的32位地址指针的段地址部分送入DS, 偏移量部分送入SI

⇒ LES 寄存器, 源操作数

与LDS相比, 仅将段地址送入ES 这一点不同

例. LEA BX, [BX+SI+0F62H]; 将源操作数的^{有效}偏移地址送往BX寄存器

LDS SI, [10H]; 从内存偏移地址10H开始的4字节地址指针中拆分出偏移量和段地址分送往SI和DS

LES DI, [BX]; 从BX指向内存地址的4字节地址指针中拆分出偏移量和段地址, 分送往DI和ES

地址传送与数据传送的区别

例. 假设某数据段定义如下

要设偏移地址则在此处加, 例如: `ORG 1000H`

不看
0000 DATA SEGMENT
00002650 标签 TABLE DW 2650H → 定义的第1个字, 值为2650H, 占
00022000 DW 2000H → 定义的第2个字, 值为2000H, 占用内存
0004 DATA ENDS
地址0002H和0003H

`MOV BX, TABLE` ①

用内存地址0000H和0001H

`LEA BX, TABLE` ②

`MOV BX, OFFSET TABLE` ③

请指出右侧指令的执行结果并说明它们之间的区别:

① 执行后, $(BX) = 2650H$

② 执行后, $(BX) = 0000$

③ 执行后, $(BX) = 0000$

① 用直接寻址的方式把变量TABLE的内容送入BX

②③ 把TABLE的偏移地址送入BX

但是, LEA指令可以使用各种存储器寻址方式, 如

`LEA BX, [DI]`

`LEA BX, TABLE[DI]`

`LEA SI, [BX+DI] ...`

都可以把计算出来的有效地址送往目的寄存器, 执行速度相对慢

而OFFSET指示符不能使用这些寻址方式, 只能作用于TABLE等简单变量或标号, 执行速度相对快

`LDS BX, TABLE` ④

`LES BX, TABLE` ⑤

④ 执行后, $(BX) = 2650H$, $(DS) = 2000H$

⑤ 执行后, $(BX) = 2650H$, $(ES) = 2000H$

执行结果?

从指定偏移地址开始取连续的双字, 4个内存单元, 前2个送往基址/变址寄存器, 后2个送往段寄存器

(6) 标志寄存器传送指令影响标志位

标志送AH指令: LAHF

$(FLAGS \text{ 的低字节}) \rightarrow (AH)$

AH送标志寄存器指令: SAHF

$(AH) \rightarrow (FLAGS \text{ 的低字节})$

标志进栈指令: PUSHF

$(SP) - 2 \rightarrow (SP)$

$(FLAGS) \rightarrow (SP+1, (SP))$; 标志寄存器的值存入栈顶2字节

标志出栈指令: POPF

$((SP)+1, (SP)) \rightarrow (FLAGS)$; 栈顶2字节的值弹出到标志寄存器
 $(SP) + 2 \rightarrow (SP)$